

classification test on new ECG PCG (128+128=256)

June 7, 2025

```
[1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

1 load data & Split

```
[8]: # Load feature and target data
features = pd.read_csv(r"D:\JN\data set\created dataset\ECG PCG\
    ↳feature(128+128=256).csv", header=None, skiprows=[0]) # Adjust header based
    ↳on your CSV format
targets = pd.read_csv(r'D:\JN\data set\data from web\Reference_aug.csv',
    ↳header=None)
```

```
[9]: features.shape
```

```
[9]: (1975, 256)
```

```
[10]: targets.shape
```

```
[10]: (1975, 2)
```

```
[11]: features.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1975 entries, 0 to 1974
Columns: 256 entries, 0 to 255
dtypes: float64(256)
memory usage: 3.9 MB
```

```
[12]: features.head(2)
```

```
[12]:    0         1         2         3         4         5         6         7         8  \
0  0.0  0.000167  0.0  0.007828  0.0  0.033072  0.055101  0.021074  0.0
1  0.0  0.000000  0.0  0.011951  0.0  0.037525  0.057435  0.023359  0.0
```

	9	...	246	247	248	249	250	251	\
0	0.023533	...	0.0	0.377506	0.103987	0.188215	0.076151	0.254665	
1	0.020270	...	0.0	0.399821	0.170468	0.241927	0.086639	0.317672	

	252	253	254	255
0	0.142509	0.215170	0.395310	0.118617
1	0.142581	0.200751	0.430879	0.154258

[2 rows x 256 columns]

```
[13]: # Extract target column
y = targets.iloc[:, -1] # Selecting the last column
X = features # All feature columns
```

```
[14]: # Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↪random_state=42)
```

2 algos

```
[15]: from sklearn.metrics import accuracy_score, classification_report,
↪confusion_matrix
```

```
[16]: ## LogisticRegression
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
# Predict on training and testing data
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Accuracy scores
train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

# Print results
print(f"Training Accuracy: {train_accuracy:.2f}%")
print(f"Testing Accuracy: {test_accuracy:.2f}%\n")

print("Classification Report (on test data):")
print(classification_report(y_test, y_test_pred))
```

Training Accuracy: 61.84%

Testing Accuracy: 58.23%

Classification Report (on test data):

	precision	recall	f1-score	support
-1	0.55	0.68	0.61	190
1	0.62	0.49	0.55	205
accuracy			0.58	395
macro avg	0.59	0.59	0.58	395
weighted avg	0.59	0.58	0.58	395

```
[17]: ## KNeighborsClassifier
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
# Predict on training and testing data
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Accuracy scores
train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

# Print results
print(f"Training Accuracy: {train_accuracy:.2f}%")
print(f"Testing Accuracy: {test_accuracy:.2f}%\n")

print("Classification Report (on test data):")
print(classification_report(y_test, y_test_pred))
```

Training Accuracy: 75.38%

Testing Accuracy: 56.20%

Classification Report (on test data):

	precision	recall	f1-score	support
-1	0.54	0.64	0.58	190
1	0.59	0.49	0.54	205
accuracy			0.56	395
macro avg	0.57	0.56	0.56	395
weighted avg	0.57	0.56	0.56	395

```
[18]: ## SVM
from sklearn.svm import SVC

model = SVC(kernel='linear')
```

```

model.fit(X_train, y_train)
# Predict on training and testing data
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Accuracy scores
train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

# Print results
print(f"Training Accuracy: {train_accuracy:.2f}%")
print(f"Testing Accuracy: {test_accuracy:.2f}%\n")

print("Classification Report (on test data):")
print(classification_report(y_test, y_test_pred))

```

Training Accuracy: 62.53%

Testing Accuracy: 57.47%

Classification Report (on test data):

	precision	recall	f1-score	support
-1	0.55	0.68	0.61	190
1	0.62	0.47	0.54	205
accuracy			0.57	395
macro avg	0.58	0.58	0.57	395
weighted avg	0.58	0.57	0.57	395

```

[19]: ## Decision Tree
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
model.fit(X_train, y_train)
# Predict on training and testing data
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Accuracy scores
train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

# Print results
print(f"Training Accuracy: {train_accuracy:.2f}%")
print(f"Testing Accuracy: {test_accuracy:.2f}%\n")

```

```
print("Classification Report (on test data):")
print(classification_report(y_test, y_test_pred))
```

Training Accuracy: 100.00%

Testing Accuracy: 60.76%

Classification Report (on test data):

	precision	recall	f1-score	support
-1	0.58	0.67	0.62	190
1	0.64	0.55	0.59	205
accuracy			0.61	395
macro avg	0.61	0.61	0.61	395
weighted avg	0.61	0.61	0.61	395

```
[20]: ## Random Forest
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
# Predict on training and testing data
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Accuracy scores
train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

# Print results
print(f"Training Accuracy: {train_accuracy:.2f}%")
print(f"Testing Accuracy: {test_accuracy:.2f}%\n")

print("Classification Report (on test data):")
print(classification_report(y_test, y_test_pred))
```

Training Accuracy: 100.00%

Testing Accuracy: 72.91%

Classification Report (on test data):

	precision	recall	f1-score	support
-1	0.70	0.77	0.73	190
1	0.77	0.69	0.72	205
accuracy			0.73	395
macro avg	0.73	0.73	0.73	395

weighted avg	0.73	0.73	0.73	395
--------------	------	------	------	-----

```
[21]: ## Naïve Bayes
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, y_train)
# Predict on training and testing data
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Accuracy scores
train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

# Print results
print(f"Training Accuracy: {train_accuracy:.2f}%")
print(f"Testing Accuracy: {test_accuracy:.2f}%\n")

print("Classification Report (on test data):")
print(classification_report(y_test, y_test_pred))
```

Training Accuracy: 62.34%

Testing Accuracy: 57.47%

Classification Report (on test data):

	precision	recall	f1-score	support
-1	0.55	0.68	0.61	190
1	0.62	0.47	0.54	205
accuracy			0.57	395
macro avg	0.58	0.58	0.57	395
weighted avg	0.58	0.57	0.57	395

```
[23]: from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_train = le.fit_transform(y_train)
y_test = le.transform(y_test)
```

```
[24]: ## xgboost
from xgboost import XGBClassifier

model = XGBClassifier()
model.fit(X_train, y_train)
# Predict on training and testing data
```

```

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Accuracy scores
train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

# Print results
print(f"Training Accuracy: {train_accuracy:.2f}%")
print(f"Testing Accuracy: {test_accuracy:.2f}%\n")

print("Classification Report (on test data):")
print(classification_report(y_test, y_test_pred))

```

Training Accuracy: 100.00%

Testing Accuracy: 73.92%

Classification Report (on test data):

	precision	recall	f1-score	support
0	0.69	0.83	0.75	190
1	0.80	0.66	0.72	205
accuracy			0.74	395
macro avg	0.75	0.74	0.74	395
weighted avg	0.75	0.74	0.74	395